TECHNICAL HANDOVER DOCUMENT | PRODUCTION READY | VITEST 29/29 PASSING

Dental Laboratory Case Management System

Comprehensive System Engineering Handover & Operational Verification Report

PROJECT TARGET	CORE STACK	VERIFICATION STATUS	HANDOVER DATE
Dental Laboratory CMS	React 19 + TypeScript + Supabase	100% Pass (Build: 0 errors)	October 2026

1. Implemented Features & Functional Scope

The Dental Laboratory Case Management System (CMS) manages the complete physical and digital lifecycle of custom dental prosthetics, restorations, and appliances. The system has been architected, coded, verified, and documented from scratch.

A. Core Case Lifecycle & State Machine

- **Atomic Monthly Case Numbering**: Server-side generation using PostgreSQL function `generate_case_number()` with row locking. Formats as `CASE-YYYYMM-NNNNN`, resets each month, guarantees collision-free concurrency, and scales past 99,999 without truncation or recycling.
- **Independent Case Traceability**: Every case maintains an independent identity, clinical prescription, and status history. Delivery batches group logistical transfers without merging individual cases.
- **Database-Enforced State Machine**: Trigger `trg_validate_case_status` validates all status transitions across all 14 statuses:
  `intake_pending → registered → awaiting_assignment → assigned → in_progress → completed → awaiting_reception_receipt → received_by_reception → awaiting_representative_pickup → with_representative → delivered` (with `on_hold`, `rework_required`, and `cancelled` branches).
- **Physical Handoff Separation**: Technician completion is strictly distinguished from reception's physical receipt. Delivery staging is decoupled from driver pickup and clinic receipt.
- **Immutable Event Logs**: `case_events` and `audit_logs` tables are strictly append-only. `UPDATE` and `DELETE` privileges have been explicitly revoked in PostgreSQL.

B. Dedicated Role-Based Workspaces

- **Owner Dashboard (`/`)**: Executive KPI metric cards, case pipeline workflow counts, overdue alerts, recent cases table, and administrative audit monitoring.
- **Receptionist Dashboard (`/`)**: Front-desk check-in, unassigned queue, one-click technician assignment modal, receipt verification, and dispatch staging.
- **Technician Workbench (`/`)**: Personal workbench showing only assigned cases, tooth numbers, shade guide, start work, pause on hold with reason, and submit to reception.
- **Representative Logistics (`/` & `/deliveries`)**: Intake batch creation, manifest management, box count discrepancy reporting, and clinic delivery confirmation.
- **Case Management (`/cases` & `/cases/:id`)**: Search by case number or patient, status filters, pagination, assignment dialogs, interactive status transitions, and chronological event timelines.
- **Case Registration Form (`/cases/new`)**: Form capturing patient details, clinic/doctor references (with inline clinic creation), tooth notation, shade guide, due dates, and priority.

- **Staff Management (`/employees`)**: Owner portal for staff invites, role modification, active/inactive toggles, password reset dispatch, and automated inspection of active cases before deactivating technicians.
- **System Audit Log (`/audit`)**: Immutable security timeline inspecting all user management actions, role updates, and system events.

2. Technology Stack & Architecture Decisions

Layer	Technology	Architectural Rationale
Frontend Framework	React 19 + TypeScript	Strict compile-time type safety, component modularity, and high rendering performance.
Build Tool & Bundler	Vite 8 + `@tailwindcss/vite`	Sub-second HMR, optimized tree-shaking producing a 558 KB production bundle (161 KB gzipped).
Styling & Design System	Tailwind CSS v4 + Design Tokens	Accessible, responsive design system with zero runtime CSS overhead and unified color variables.
State & Cache	TanStack Query v5	Automatic background refetching, query deduplication, and mutation cache invalidation.
Routing & Guards	React Router v7	Declarative route protection (`ProtectedRoute`) and role-based entrypoints (`DashboardRouter`).
Database & Auth	Supabase (PostgreSQL 15+)	Full relational integrity, atomic sequence counters, Row-Level Security, zero server maintenance costs.
Privileged Server Logic	Supabase Edge Functions	Deno runtime for administrative operations requiring the service-role key (user provisioning, session revocation).
Automated Testing	Vitest 5.0.4 + JSDOM	Fast ESM test runner integrated with Vite path aliases; 29/29 tests pass cleanly.

3. Database Schema & Relational Architecture

The database schema enforces referential integrity through foreign keys, domain check constraints, and Row Level Security across all entities:

Table Name	Primary / Foreign Keys	Purpose & Security Controls
`profiles`	PK: `id` (UUID) FK: `auth.users(id)`	Internal laboratory staff profiles. Stores role, status, email. Guarded by `protect_last_active_owner` trigger.
`dentists`	PK: `id` (UUID) FK: `created_by`	External dental clinics and practitioners. External reference entity (no system login).
`monthly_case_counters`	PK: `year_month` (TEXT)	Atomic monthly counter (YYYYMM). Updated via `SELECT FOR UPDATE` in `generate_case_number()`. Direct client access revoked.
`cases`	PK: `id` (UUID) UK: `case_number` FK: `dentist_id`, `assigned_technician_id`	Principal case entity. Stores patient demographics, clinical tooth/shade specs, due date, and workflow state. Validated by transition trigger.
`delivery_batches`	PK: `id` (UUID) UK: `batch_reference` FK: `representative_id`	Physical transfer batches. Tracks intake/return status, expected count vs. actual count, and discrepancy remarks.
`batch_cases`	PK: `id` (UUID) FK: `batch_id`, `case_id`	Many-to-many junction joining batches and individual cases. Ensures cases remain independently traceable.
`case_events`	PK: `id` (UUID) FK: `case_id`, `actor_id`	**IMMUTABLE** operational event log. Records every status transition, handoff, and internal note. UPDATE and DELETE revoked.
`technician_assignments`	PK: `id` (UUID) FK: `case_id`, `technician_id`	Historical assignment records. Tracks active technician, assignment timestamp, receipt acknowledgement, and reassignment reasons.
`audit_logs`	PK: `id` (UUID) FK: `actor_id`	**IMMUTABLE** administrative security log. Captures user provisioning, role adjustments, and deactivations. UPDATE and DELETE revoked.

4. Available Roles & Permissions Matrix (RBAC)

Action / Resource	Owner	Receptionist	Representative	Technician
View All Cases	Full	Full	Full	Assigned Only
Register New Case & Clinic	✓ Allowed	✓ Allowed	✗ Denied	✗ Denied
Assign / Reassign Technician	✓ Allowed	✓ Allowed	✗ Denied	✗ Denied
Start Work / Mark Completed	✓ Allowed	✗ Denied	✗ Denied	✓ Assigned Only
Submit to Reception (Stage D)	✓ Allowed	✗ Denied	✗ Denied	✓ Assigned Only
Acknowledge Physical Receipt	✓ Allowed	✓ Allowed	✗ Denied	✗ Denied
Pick Up Return Batch (Stage E)	✓ Allowed	✗ Denied	✓ Allowed	✗ Denied
Confirm Clinic Delivery (Stage F)	✓ Allowed	✗ Denied	✓ Allowed	✗ Denied
Put On Hold / Request Rework	✓ Allowed	✓ Allowed	✗ Denied	✓ Assigned Only
Manage Employees & Permissions	✓ Allowed	✗ Denied	✗ Denied	✗ Denied
View System Security Audit Logs	✓ Allowed	✗ Denied	✗ Denied	✗ Denied
Modify / Delete Historical Events	✗ **FORBIDDEN**	✗ **FORBIDDEN**	✗ **FORBIDDEN**	✗ **FORBIDDEN**

5. Actual Build, Test & Security Results

A. Automated Test Execution (Vitest 5.0.4)

The entire test suite passed with 29 passing assertions in 1.64 seconds:

```
RUN  v5.0.4
✓ src/test/handoffs.test.ts (3 tests)
✓ src/test/caseNumber.test.ts (5 tests)
✓ src/test/stateMachine.test.ts (4 tests)
✓ src/test/employeeAdmin.test.ts (4 tests)
✓ src/test/permissions.test.ts (8 tests)
✓ src/test/utils.test.ts (5 tests)

Test Files  6 passed (6)
     Tests  29 passed (29)
  Duration  1.64s
```

B. Production Build Verification

TypeScript compiler checks and Vite asset optimization passed cleanly with zero errors:

```
> tsc -b && vite build
✓ 2041 modules transformed.
dist/index.html                              0.79 kB │ gzip:  0.39 kB
dist/assets/index-DT4Dbr8a.css              35.94 kB │ gzip:  7.74 kB
dist/assets/rolldown-runtime-CbXtAM7H.js     0.58 kB │ gzip:  0.36 kB
dist/assets/index-Wqh2Kh5S.js                0.95 kB │ gzip:  0.57 kB
dist/assets/query-DR6sRXM9.js               43.00 kB │ gzip: 13.02 kB
dist/assets/supabase-1yby8ldT.js           214.04 kB │ gzip: 55.06 kB
dist/assets/vendor-CotpznSL.js             264.82 kB │ gzip: 84.68 kB
✓ built in 539ms
```

C. Security & Authorization Safeguards Verified

- **Last Active Owner Invariant**: Verified that trigger `protect_last_active_owner` rejects deactivating, demoting, or deleting the sole active Owner.
- **Technician IDOR Protection**: Verified that technicians cannot read, modify, or advance cases assigned to other technicians.
- **Deactivation & Reassignment Procedure**: Verified that function `reassign_and_deactivate_technician` detects open cases, reassigns them to an active replacement, creates historical audit events, and disables user sessions.
- **Event Log Immutability**: Verified that `UPDATE` and `DELETE` operations are revoked on `case_events` and `audit_logs`.

6. Local Execution & Commands

```
# 1. Install dependencies
npm install

# 2. Run automated test suite
npm test

# 3. Start local development server (Vite on http://localhost:5173)
npm run dev

# 4. Compile for production
npm run build
```

7. Environment Variables Configuration

> **Security Policy:** The browser bundle contains only public anonymous credentials. The `SUPABASE_SERVICE_ROLE_KEY` is never committed, bundled, or exposed to the frontend.

```
# Required in .env for frontend development:
VITE_SUPABASE_URL=https://[YOUR_PROJECT_ID].supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# Server-side ONLY (Supabase Edge Function secrets):
# SUPABASE_SERVICE_ROLE_KEY=your-service-role-key-here
```

8. Database Migration & Initial Owner Provisioning

1. Log into the **Supabase Dashboard** → Open the **SQL Editor**.
2. Execute migration files in order:
   - `supabase/migrations/001_initial_schema.sql` (Custom enums, tables, sequences, functions, triggers, and RLS).
   - `supabase/migrations/002_owner_protections.sql` (Owner guards, outstanding checks, and atomic reassignment).
   - *(Optional Demo Seed)* `scripts/seed_demo_data.sql` (Populates sample dentists and test cases).
3. **Initial Owner Provisioning (Zero-Backdoor Procedure)**:
   - Under **Authentication > Users**, click **"Add user"** → **"Create user"**.
   - Enter the Owner's email (e.g. `owner@dentallab.com`) and a temporary password. Check **"Auto Confirm User"**.
   - In the SQL Editor, grant the initial Owner role:

     ```
     UPDATE profiles SET role = 'owner', status = 'active' WHERE email = 'owner@dentallab.com';
     ```

   - The Owner can now sign in at `/login` and manage staff from `/employees`.

9. Deployment & Free-Tier Operational Guarantees

The system is designed to operate completely within free-tier service limits:

- **Hosting (Vercel / Netlify)**: Zero hosting costs using static SPA deployment with rewrite rule pointing to `/index.html`.
- **Database Storage (500 MB Quota)**: Supabase free tier provides 500 MB of PostgreSQL storage, accommodating approximately **250,000 cases**. If the quota is approached, completed cases can be exported using the provided backup scripts.
- **Inactivity Pausing (7 Days)**: Supabase free projects pause after 7 consecutive days of zero API requests. In daily dental lab operations, pausing will not occur. If paused during holiday shutdowns, unpausing takes 1–2 minutes from the Supabase dashboard.
- **Off-Site Backups**: Run the automated backup scripts to preserve off-site database copies:
  Windows: `.\scripts\backup.ps1 -DatabaseUrl "postgresql://postgres:[PASSWORD]@[HOST]:5432/postgres"`
  Linux/macOS: `./scripts/backup.sh "postgresql://postgres:[PASSWORD]@[HOST]:5432/postgres"`

10. Known Limitations & Future Recommendations

- **Case File Attachments**: Kept optional in Phase 1 to prevent exceeding the 1 GB Supabase free storage limit. Prescription slips and clinical notes are currently captured in structured text fields.
- **Email Rate Limits**: Built-in Supabase Auth email service is limited to 30 emails/hour on the free tier. For large-scale labs with frequent invitations, connecting a custom SMTP service (e.g. Resend free tier) is recommended.

11. Complete Documentation Index

Document	File Path	Content Summary
Project Specification	`docs/PROJECT_SPECIFICATION.md`	Business context, roles, core requirements, and acceptance criteria.
Architecture Specification	`docs/ARCHITECTURE.md`	Trust boundaries, 3-tier design, data flow, and technology decisions.
Database Schema	`docs/DATABASE_SCHEMA.md`	Complete data dictionary, column types, constraints, and indexes.
Entity-Relationship Diagram	`docs/ERD.md`	Mermaid ER diagram showing entity relationships and foreign keys.
Permissions Matrix	`docs/PERMISSIONS_MATRIX.md`	RBAC matrix across all 4 roles and negative authorization rules.
Case Workflow	`docs/CASE_WORKFLOW.md`	State machine lifecycle, handoffs, and valid transition rules.
Security Architecture	`docs/SECURITY.md`	OWASP threat model, RLS policies, IDOR defenses, and secrets isolation.
Security Checklist	`docs/SECURITY_CHECKLIST.md`	Verification checklist tracking implemented OWASP controls.
Agent Skills Report	`docs/AGENT_SKILLS.md`	Skills discovery, evaluation, and usage report.
Test Plan & Results	`docs/TEST_PLAN.md`	Test strategy, suite breakdown, and 29/29 passing test results.
Deployment Guide	`docs/DEPLOYMENT.md`	Setup, initial Owner provisioning, backups, and free-tier quotas.